Автономная некоммерческая организация высшего образования
«Санкт-Петербургский институт экономики и управления»

Кафедра **гуманитарных, математических и естественно научных дисциплин**

ЭССЕ

Дисциплина(модуль)/предмет **МДК.02.02**

Тема: **Роль процессного подхода и международных стандартов (ISO/IEC 12207) в минимизации рисков при проектировании жизненного цикла ПО**

ВАРИАНТ № **3**

Студент (ка)/учащийся: **Браунков Матвей Константинович** / _____ /
Ф. И. О. подпись

Группа / Класс И14-23-9

Руководитель: **Чудинова А.А.** /_____ /
Должность, степень, Фамилия и инициалы подпись

Дата представления работы «____» _____ 202__г.
Работа аттестована с оценкой _____
прописью

Санкт-Петербург
2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....3
ОСНОВНАЯ ЧАСТЬ.....4
ЗАКЛЮЧЕНИЕ.....7
СПИСОК ЛИТЕРАТУРЫ.....8

ВВЕДЕНИЕ

Разработка программного обеспечения остаётся областью с высокой долей проектов, которые выходят за пределы срока и бюджета или поставляются с дефектами. Одной из главных причин является хаотичная организация работ: требования фиксируются устно, этапы не разделены, а проверка качества откладывается на конец проекта. В таких условиях риски не исчезают, а накапливаются и проявляются в самый дорогой момент – при сборке модулей в единую систему.

Актуальность темы определяется тем, что международные и национальные стандарты, прежде всего ГОСТ Р ИСО/МЭК 12207, предлагают проверенный способ превратить неупорядоченную разработку в управляемый процесс [1], [2]. Проблема заключается в том, чтобы определить и обосновать, за счёт чего именно процессный подход снижает риски на стадиях верификации и интеграции модулей.

Тезис работы: переход от хаотичной разработки к процессной модели жизненного цикла по ISO/IEC 12207 снижает проектные риски, потому что делает их видимыми на ранних этапах и закрепляет обязательные проверки перед интеграцией. В работе рассматриваются особенности хаотичной разработки, структура стандарта, стандартизированные верификация и интеграция, а также ограничения подхода и его экономическая целесообразность для реального проекта.

ОСНОВНАЯ ЧАСТЬ

1 Хаотичная разработка и её риски

Хаотичной называют разработку по принципу «написать и исправить» (code and fix), при которой отсутствуют формализованные процессы. Результат зависит от отдельных исполнителей, а знания о системе не фиксируются. Ф. Брукс показал, что добавление людей в запаздывающий проект лишь увеличивает затраты на коммуникацию, а основная сложность связана с согласованием частей системы [3]. Для хаотичных проектов характерны нечёткие требования, отсутствие трассируемости, поздняя интеграция и тестирование «в конце». При интеграции «большим взрывом» все модули собираются одновременно, поэтому ошибка в интерфейсе обнаруживается поздно, а её причину трудно локализовать. Стоимость исправления дефекта растёт по мере продвижения проекта по жизненному циклу [4]. Кроме того, при хаотичной разработке отсутствует базовая линия конфигурации: невозможно определить, какая версия модуля прошла проверку, поэтому ошибки воспроизводятся нестабильно, а изменения одного разработчика неожиданно нарушают работу другого [7]. Совокупность этих факторов делает риски неуправляемыми: их нельзя оценить и поставить на контроль, пока они не превратились в сбой.

2 Процессный подход и структура ISO/IEC 12207

Процессный подход рассматривает деятельность как набор взаимосвязанных процессов с входами, выходами и ответственными. Стандарт ГОСТ Р ИСО/МЭК 12207 (ISO/IEC/IEEE 12207) определяет единый перечень процессов жизненного цикла программных средств [1], [2]. Он объединяет процессы соглашения, организационного обеспечения проекта, управления проектом и технические процессы: анализ требований, проектирование архитектуры, реализацию, интеграцию, верификацию, валидацию, ввод в эксплуатацию и сопровождение. К ним добавляются процессы управления конфигурацией, обеспечения качества и измерения. Важная особенность стандарта: он описывает, какие процессы необходимы, но

не навязывает модель жизненного цикла. Организация может применять каскадную, итеративную или гибкую модель, адаптируя (tailoring) набор процессов под проект. Последовательное улучшение процессов соотносится с уровнями зрелости модели CMM [5]. Процессы поддержки обеспечивают проверяемость: управление конфигурацией фиксирует версии, обеспечение качества проверяет соблюдение процедур, а процесс управления рисками требует выявлять, оценивать и отслеживать угрозы на протяжении всего проекта. Тем самым управление рисками становится встроенной частью жизненного цикла, а не отдельным разовым мероприятием.

3 Стандартизированные верификация и интеграция модулей

Верификация отвечает на вопрос «правильно ли создаётся продукт», то есть соответствует ли результат этапа его исходным требованиям. Валидация проверяет, что создан именно тот продукт, который нужен заказчику. Для верификации применяются инспекции и code review, статический анализ и модульное тестирование; при этом уровень строгости определяется целостностью системы [6]. Интеграция в процессной модели выполняется поэтапно по заранее спроектированной архитектуре: сначала проверяются отдельные модули, затем их взаимодействие через интерфейсы. Современным развитием этого подхода является непрерывная интеграция, при которой автоматическая сборка и тесты запускаются при каждом изменении кода. Ключевые инструменты снижения риска – критерии входа и выхода из этапа, трассируемость требований к тестам и фиксация версий в системе управления конфигурацией. Благодаря этому дефект интерфейса обнаруживается на уровне пары модулей, а не всей системы. Показательно сравнение двух ситуаций. При хаотичной сборке девять модулей объединяются в последний день, и сбой может быть вызван любым из 36 возможных парных взаимодействий. При поэтапной интеграции каждое новое подключение проверяется отдельным набором тестов, поэтому область поиска причины ограничена одним модулем.

4 Ограничения и экономический эффект

Внедрение процессов требует затрат на обучение, документирование и контроль. При формальном подходе возникает бюрократизация: документы создаются ради отчётности, а не для управления риском. Для небольших команд стандарт избыточен без адаптации, поэтому существуют облегчённые профили для малых организаций. Экономический эффект проявляется в сокращении доли переделок, предсказуемости сроков и снижении числа дефектов в эксплуатации. Эти затраты окупаются, когда стоимость исправления поздних ошибок превышает стоимость предотвращения [4]. Дополнительное условие успеха – культура качества: стандарт даёт структуру, но выполняют его люди. Поэтому внедрение целесообразно начинать с минимального набора практик и расширять его по мере роста проекта.

ЗАКЛЮЧЕНИЕ

Анализ показал, что хаотичная разработка накапливает риски: неточные требования, поздняя проверка и интеграция «большим взрывом» приводят к дорогим дефектам и непредсказуемым срокам. Процессный подход на основе ГОСТ Р ИСО/МЭК 12207 заменяет случайные действия набором определённых процессов с входами, выходами и ответственными, что делает риски видимыми и управляемыми [1], [2].

Наибольший вклад в снижение рисков дают стандартизированные верификация и поэтапная интеграция: проверки выполняются до перехода к следующему этапу, интерфейсы модулей тестируются раньше, а изменения отслеживаются через управление конфигурацией. Автоматизация сборки и тестирования делает эти практики постоянными.

Эффективность подхода ограничивается риском формализма и затратами на внедрение, поэтому процессы необходимо адаптировать к масштабу проекта. Тезис работы подтверждается: переход к стандартизированным процессам повышает предсказуемость и качество результата, если стандарт применяется осмысленно, а не как набор формальных требований. Практическая значимость вывода состоит в том, что даже при ограниченных ресурсах проект выигрывает от минимального набора обязательных практик: письменных требований, контроля версий, обязательного ревью кода и автоматизированных тестов перед каждой интеграцией.

СПИСОК ЛИТЕРАТУРЫ

1. ГОСТ Р ИСО/МЭК 12207-2010. Информационная технология. Системная и программная инженерия. Процессы жизненного цикла программных средств. – М.: Стандартинформ, 2011. – 105 с.

2. ISO/IEC/IEEE 12207:2017. Systems and software engineering – Software life cycle processes. – Geneva: ISO, 2017.

3. Brooks F.P. The Mythical Man-Month: Essays on Software Engineering. Anniversary Edition. – Boston: Addison-Wesley, 1995. – 322 p.

4. Sommerville I. Software Engineering. 10th ed. – Harlow: Pearson, 2015. – 816 p.

5. Paulk M.C., Curtis B., Chrissis M.B., Weber C.V. Capability Maturity Model for Software, Version 1.1. Technical Report CMU/SEI-93-TR-024. – Pittsburgh: Software Engineering Institute, 1993.

6. IEEE 1012-2016. IEEE Standard for System, Software, and Hardware Verification and Validation. – New York: IEEE, 2016.

7. McConnell S. Code Complete. 2nd ed. – Redmond: Microsoft Press, 2004. – 960 p.